基于混合效应模型的 NIPT 时点优化与胎儿异常判定

摘　要

关键字： 无创产前检测　混合效应模型　优化模型　Z 值判定　染色体异常筛查

一、 问题重述

1.1 问题背景

无线电频谱管理的关键任务之一是无线电干扰源的定位与清除，通常分为初步检测、区域性排查与精确定位三个基本阶段。前两个阶段通过固定监测站，移动监测车或无人机等设备采集并分析大范围信号以确定干扰源的分布区域。然而，由于环境遮挡、低功率干扰源等因素的影响，精准定位仍需技术人员携带测向机，结合信号场强变化和方向特征进行现场排查。为提高效率，某公司拟研发搭载于机器狗的自动定位清除系统，在危险环境中代替人工完成干扰源的定位与清除。

1.2 问题要求

本研究需结合题设条件，建立数学模型以系统研究以下四个问题：

1. 根据若干检测点坐标及测得的同一干扰源的示向度，通过交会定位法确定多边形区域，设计算法计算该区域任意两点间的最大距离并判断以该距离为直径的圆能否完全覆盖此区域。
2. 给定一全向干扰源在某检测点处的示向度，试给出第二个检测点的选择策略并确定有利于提高定位效果的候选区域。
3. 针对目标区域内数量未知的全向干扰源，派出一机器狗从原点出发，对干扰源进行定位与清除。在确保所有干扰源都能被清除的前提下，试设计机器狗的搜索定位与清除算法，尽量缩短任务完成时间。
4. 在问题（3）的基础上，进一步考虑全向与定向干扰源共存的情形，其中干扰源数量与定向干扰源的发射方向均未知。针对这一情况调整机器狗的搜索定位与清除算法，在确保全部干扰源被清除的前提下，尽量缩短任务完成时间。

其中，问题（3）与问题（4）均需通过模拟器开展演练及三次正式测试，验证所设计算法的有效性。

二、 问题分析

本题主要分为基于交会定位的区域直径计算与圆覆盖问题、以提高精度为目标的第二检测点选址优化问题，以及多干扰源的搜索定位与清除策略优化问题。

根据题目与附录信息，首先将示向度误差、信号接收范围等条件转化为干扰源位置的约束，以建立交会定位模型；其次，综合考虑包括机器狗移动、频道切换、检测与清除在内的各项时间成本，确定策略优化的核心目标；最后通过模拟器进行测试与演练，

根据成功清除比例与平均清除时间对策略进行评价，并完成正式测试。

2.1 问题一的分析

问题一要求我们根据检测点坐标及示向度，绘制多边形定位区域，给出计算该区域直径的算法，并探究以该直径为直径的圆能否覆盖定位区域。

根据题给信息，在考虑进 $\pm 1^{\circ}$ 的误差及示向度后，每个检测点可唯一确定一个包含干扰源的角域，多个检测点角域的交集即为定位区域。由于该区域具有凸性，其区域内任意两点之间距离的最大值必由一对最远顶点确定，考虑到穷举法遍历所有顶点对的时间复杂度较高，可利用定位区域的凸性，采用旋转卡壳算法进行优化计算。

对于圆覆盖问题，只由直径的长度大于等于区域内任意两点的距离不能直接推断同直径圆能够覆盖整个区域。因此，需以直径中点为圆心、区域直径的一半为半径，依次检验各顶点是否均位于圆内，并结合该最小包围圆与反例的特点分析给出结论。

2.2 问题二的分析

三、 模型假设

为简化问题，本文做出以下假设：

1.

2.

3.

四、 符号说明

符号	说明	单位
FF	胎儿 Y 染色体浓度	–
GA	孕周	周
BMI	身体质量指数	$\mathrm{kg/m^2}$
y	$\mathrm{logit}(FF)$	–
$\beta_0, \beta_1, \ldots$	固定效应系数	–
u_i, b_i	随机效应	–
σ^2	方差	–
$p(t, b)$	达标概率	–
θ	浓度阈值（0.04）	–
$T_*(b)$	最早达标时间	周
κ	误差放缩因子	–
r_{fail}	检测失败风险	–
$\phi(t)$	临床时间风险	–
Z	Z 值	–
q	FDR 校正后的 q 值	–

五、 问题一模型的建立与求解

5.1 基于测向误差角域的交会定位模型

针对问题一，本文将测向误差转化为关于干扰源位置的约束，通过多角域的交集构造定位区域，再利用凸多边形的几何性质计算其直径。这里的定位区域采用附录 2 所述的示向度角域交集，不额外叠加其他可能会引入其他边界的可行域模型。

设对同一干扰源共进行 k 次有效测向，第 i 个检测点为 $S_i = (x_i, y_i)$，示向度为 θ_i，且已知误差为 $\varepsilon = 1^\circ$。假定将检测点坐标作为已知输入，测向信息对应同一位置不变的干扰源，且真实方向与示向度的圆周角差不超过 ε。以上约束不要求误差相互独立，也不要求其服从均匀分布。本文后续统一使用 k 表示检测点数，h 表示定位区域凸多边形顶点数，长度单位为米。

令待定位点为 $X = (x, y)$，定义

$$u_i = (\cos\theta_i, \sin\theta_i), \qquad d_i = X - S_i, \qquad t = \tan\varepsilon. \tag{1}$$

记二维叉积为 $a \times b = a_x b_y - a_y b_x$，则第 i 次测向所限定的角域为

$$W_i = \{X : u_i \times d_i - t(u_i \cdot d_i) \le 0, \quad -u_i \times d_i - t(u_i \cdot d_i) \le 0\}. \tag{2}$$

式(2)等价于 $|u_i \times d_i| \le t(u_i \cdot d_i)$。由于 $t > 0$，两式相加即可推出 $u_i \cdot d_i \ge 0$，因此该表示已经排除了反向角域。

综合全部测向信息，定义

$$P_k = \bigcap_{i=1}^{k} W_i = \{X \in \mathbb{R}^2 : AX \le b\}. \tag{3}$$

为所求的多边形定位区域。

由于每个角域均为两个闭半平面的交集，结合凸集的性质可知，P_k 是闭凸集。满足题设误差界的干扰源真实位置 G 属于每个 W_i，从而有 $G \in P_k$。区域非空且有界时，可定义并取到其直径

$$D(P_k) = \max_{X,Y \in P_k} \|X - Y\|_2. \tag{4}$$

与题中定义相同，该量描述了区域内所有相容位置间分离距离的最大值，即为所求直径的数学表示。

5.2 定位区域的判定与顶点构造

将各检测点的角域约束汇总后，定位区域可统一表示为

$$P_k = \left\{X \in \mathbb{R}^2 \mid AX \le b\right\}.$$

显然，每一个不等式都代表一个半平面，定位区域就是这些半平面的交集。为了计算区域直径，我们应首先判断交集的状态，进而确定其边界顶点。

首先，检查交集是否为空，若交集为空集，则说明当前测向数据指向的定位区域无法对应同一个干扰源位置，此时无法构造有效定位区域。

若交集非空，还需判断其是否有界。直观上，若存在一个方向，使某个可行点沿该方向移动任意远后仍满足全部约束，则区域会沿该方向无限延伸。当非空区域存在这样无限延伸的非零方向时为无界区域，显然不具有有限直径；不存在时，区域有界，可以继续构造其顶点。

对于非空有界的交集，可采用边界交点枚举法确定定位区域。具体步骤如下：

1. **计算边界交点**。将每个半平面对应的边界写成直线方程，逐一计算两条非平行边界直线的交点。
2. **筛选可行交点**。将交点代入全部半平面约束，只有满足所有不等式的交点才予以保留。

3. **整理区域边界。**对可行点使用 Andrew 单调链算法求凸包，删除重复点和边上的冗余共线点，得到按逆时针顺序排列的顶点

$$V_0, V_1, \ldots, V_{h-1}.$$

以上方法能够完整确定定位区域：首先，半平面交集的每个顶点都是至少两条非平行约束边界的交会，因此会出现在枚举结果中；同时，只有满足全部约束的点会被保留下来，因此不会引入区域外的点。由于定位区域具有凸性，这些交点的凸包恰好就是原定位区域。

每次测向提供两条非冗余角域边界，因此，对于非退化的有界定位多边形，其顶点数满足

$$h \le 2k.$$

实际顶点数可能更少，因为部分测向约束不会形成最终区域的边界。

对于一些特殊情况：若区域退化为单点，其直径直接取 0；若退化为线段，则取两端点间距。

对于具有三个及以上顶点的凸多边形，将上述有序顶点序列输入旋转卡壳算法，继续求解区域直径。

5.3 区域直径的有限化与旋转卡壳求解

5.3.1 将区域直径转化为顶点间的最大距离

在上一小节中，我们得到了定位区域的有序顶点 $V_0, V_1, \ldots, V_{h-1}$。由上文可知，对于任意非空有界定位区域 P, 所求计算区域直径的数学表示为

$$D(P) = \max_{X,Y \in P} \|X - Y\|_2. \tag{5}$$

这里的 X 和 Y 代表定位区域内的任意候选位置，可以位于区域内部、边界或顶点。显然，直接搜索区域内的全部点不可行，因此需要利用凸性将搜索范围缩小到有限个顶点。

由于 P 是凸多边形，因此其中任意点 $X, Y \in P$ 都可以用各顶点的加权平均进行表示，即：

$$X = \sum_{i=0}^{h-1} \alpha_i V_i, \qquad Y = \sum_{j=0}^{h-1} \beta_j V_j, \tag{6}$$

其中 $\alpha_i, \beta_j \geq 0$，且 $\sum_i \alpha_i = \sum_j \beta_j = 1$。因此，

$$\begin{aligned} \|X - Y\|_2 &= \left\| \sum_{i,j} \alpha_i \beta_j (V_i - V_j) \right\|_2 \\ &\leq \sum_{i,j} \alpha_i \beta_j \|V_i - V_j\|_2 \\ &\leq \max_{i,j} \|V_i - V_j\|_2. \end{aligned} \tag{7}$$

也就是说，区域内任意两点之间距离的最大值就是最远顶点对之间的距离。由于顶点本身也是区域的一部分，故区域直径可表示为

$$D(P) = \max_{0 \leq i,j < h} \|V_i - V_j\|_2. \tag{8}$$

显然，只需在所有顶点对间寻找距离最远的顶点对，便可得到整个连续区域的直径。穷举全部 $\binom{h}{2}$ 个顶点对需要 $O(h^2)$ 时间，下面利用旋转卡壳进行优化，进一步减少搜索量。

5.3.2 平行支撑线与对踵点对

设一条与凸多边形接触的直线使整个多边形都位于直线的同一侧，这样的直线称为支撑线。当两条互相平行的支撑线夹住多边形时，两条线上的接触点构成**对踵点对**。旋转卡壳的基本思想是让这两条支撑线围绕多边形转动，并随转动更新接触顶点。

5.3.3 利用三角形面积推进支撑点

对逆时针排列的顶点，固定当前边 V_iV_{i+1}，定义

$$F_i(j) = (V_{i+1} - V_i) \times (V_j - V_i), \tag{9}$$

下标按模 h 循环计数。显然，$F_i(j)$ 是三角形 $V_iV_{i+1}V_j$ 面积的两倍，即

$$F_i(j) = \|V_{i+1} - V_i\|_2 \ \mathrm{dist}(V_j, L_i), \tag{10}$$

其中 L_i 是当前边所在的直线。由于 V_iV_{i+1} 的长度一定，面积最大的顶点也就是距离 L_i 最远的顶点，即对侧支撑线的接触点。

在计算过程中，逐次比较 $F_i(j+1)$ 与 $F_i(j)$：如果前者较大，就把对侧顶点从 V_j 修改为 V_{j+1}，直到面积不再增加。这里最大化的是点到当前边所在直线的距离，并不直接等于区域直径；仍需计算相应顶点对的欧氏距离。

随着凸多边形的边方向沿边界有序变化，对应的支撑位置也按边界顺序变化。因此，在当前边移到下一条边时，对侧支撑点可以从已有位置继续向前搜索，无须回到起点，这一特点极大地减少了每条边的遍历次数。

若 $F_i(j+1) = F_i(j)$ 且两者均达到最大值，则对侧支撑线同时经过两个相邻顶点，即与一条边重合。此时需要把两个对侧端点均纳入候选，完整处理平行边情形。

5.3.4 完整算法流程

综合以上描述，完整的算法流程如下：

1) 若区域为单点，返回 $D = 0$；若区域为线段，返回两端点距离。对 $h \geq 3$ 的凸多边形，初始化对侧指针 $j = 1$ 和最大距离平方 $M = 0$。
2) 依次处理各边 V_iV_{i+1}。当 $F_i(j+1) > F_i(j)$ 时推进 j，直至到达当前边的对侧支撑位置。
3) 分别计算 $\|V_i - V_j\|_2^2$ 和 $\|V_{i+1} - V_j\|_2^2$，与 M 比较并保存较大值及其端点。
4) 若对侧出现等面积的相邻顶点，则进一步检查 $\|V_i - V_{j+1}\|_2^2$ 和 $\|V_{i+1} - V_{j+1}\|_2^2$，更新最优结果。
5) 扫描全部边后，输出 $D = \sqrt{M}$ 以及对应最远点对。

5.4 计算结果与分析

由于本题未提供检测点数据，因此本文自行构建符合条件的算例进行验证。预设干扰源位置为 $G = (150, -250)$，根据检测点与 G 的位置关系生成示向度。G 仅用于生成验证数据，求解时只输入检测点坐标与示向度；具体输入见支撑材料中的算例文件。

表 **1**　典型布设下的定位区域直径

检测布设	检测点数 k	顶点数 h	直径 D/m
两检测点分别位于源的正东方和正北方	2	4	59.262511
于源的西南方向再增加一个检测点	3	6	59.262511
两检测点与源的夹角为 5°	2	4	1190.734431
两检测点均位于源的正东方且距离不同	2	—	无有限直径

表1中各有界算例的旋转卡壳结果均与对全部顶点进行穷举计算的结果一致。在 1000 个随机凸多边形的对照测试中，两种方法的最大数值差同样为 0，表明在此测试范围内直径的计算结果是可靠的。

不同布设的交会结果存在明显差异。新增第三站后，区域顶点数由 4 增至 6，但原最远点对仍被保留，因此直径未变。夹角较小的算例中直径显著增大，同方位算例则出现无界区域，说明定位具有不确定性且检测点的空间布设有关。

5.5 定位区域的同直径圆覆盖分析

给定前文求得的非空有界定位区域 P 的直径 D 及构成直径的最远顶点 A, B。以下讨论是否存在直径为 D 的圆覆盖整个区域。值得注意的是，当区域退化为单点时，半径为零的圆盘即可覆盖；下文仅考虑 $D > 0$ 的情形。

5.5.1 圆心确定与顶点覆盖判据

设某个半径为 $D/2$、圆心为 O 的圆盘能够覆盖 P，显然它同时包含了 A, B。

由于题目要求 O 在线段 AB 上，且到两端点的距离均为 $D/2$，即 A, B 是圆的直径，因此圆心只能取

$$O = \frac{A + B}{2}. \tag{11}$$

这说明，同直径圆能否覆盖区域并不是任意选取圆心的问题：若以 O 为圆心的圆盘不能覆盖，则移动圆心也无法在保持直径 D 的同时实现覆盖。

对区域顶点 $V_0, \ldots, V_{h-1}$，定义

$$\rho = \max_{0 \le i < h} \|V_i - C\|_2, \qquad \Delta = \rho - D/2. \tag{12}$$

显然，当任意到圆心的距离都小于等于半径的时候，同直径圆盘可以覆盖 P，即

$$\boxed{\text{存在同直径圆盘覆盖} P \quad \Longleftrightarrow \quad \rho \le D/2.} \tag{13}$$

也就是说，若 $\Delta > 0$，ρ 对应的顶点就是不能覆盖的直接证据。在已知直径端点的前提下，只需遍历全部顶点即可检验，检验复杂度为 $O(h)$。

5.5.2 由测向角域构造反例

为进一步判断是否能完全覆盖，构造三次测向形成等边三角形的反例。设干扰源真实位置为原点，检测点及示向度如表2所示，各测向误差均为 1°。

表 **2**　等边三角形反例的测向输入（坐标单位：米）

检测点	x_i	y_i	示向度/(°)
S_1	500	$500\sqrt{3} + 20/\sqrt{3}$	241
S_2	-1010	$-10/\sqrt{3}$	1
S_3	510	$-510\sqrt{3} + 20/\sqrt{3}$	121

通过交会定位法得到角域交集的三个顶点分别为

$$A = \left(-10, -\frac{10}{\sqrt{3}}\right), \quad B = \left(10, -\frac{10}{\sqrt{3}}\right), \quad T = \left(0, \frac{20}{\sqrt{3}}\right) \tag{14}$$

干扰源位于这三个顶点围成的多边形定位区域内，距各检测点约为 1000 米，选择 1200 米有效接收半径即可满足题设接收条件。

该三角形边长为 20 米，故 $D = 20$ 米。取底边 AB 为直径端点，候选圆的圆心为 $C = (0, -10/\sqrt{3})$、半径为 10 米，而

$$\|T - C\|_2 = 10\sqrt{3} \approx 17.3205 > 10. \tag{15}$$

显然点 T 超出了现有圆盘约 7.3205 米，如图1所示。结合圆心的唯一性，任意直径为 20 米的圆盘都无法完全覆盖该定位区域。

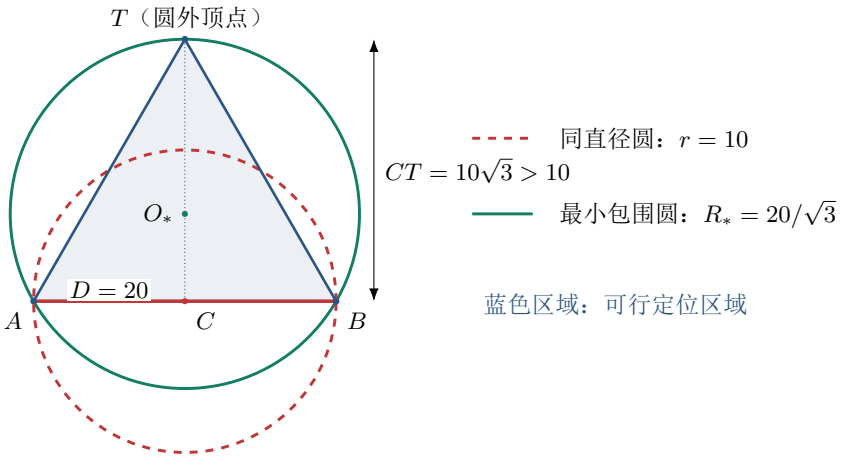


图 **1**　等边三角形反例：红色虚线圆盘不能覆盖顶点 T，绿色实线为最小包围圆。

5.5.3　最小包围圆解释与数值核验

记最小包围圆半径为 R_*。设计程序枚举候选圆并检查其是否包含全部顶点，得到表3中的结果。

表 **3**　区域直径与最小覆盖圆直径的比较（单位：米）

算例	D	$2R_*$	同直径圆能否覆盖
等边三角形反例	20.000000	23.094011	否
四边形反例	34.514803	34.532121	否
四边形对照例	49.385426	49.385426	是

通过观察与验证，以上反例具有共同的几何特征：其最小包围圆均由三个支撑顶点构成的锐角三角形的外接圆确定。其中，等边三角形由自身三个顶点支撑；四边形则由其中三个顶点支撑，剩余顶点位于圆内。

对照例则在数值容差下满足 $2R_* = D$，即能够被同直径圆完全覆盖。计算采用 10^{-7} 米的距离容差，两反例的直径差均明显大于该容差。

因此，以定位区域直径为直径的圆不一定能覆盖该区域，应结合顶点覆盖判据判断。

六、 模型的评价与推广

6.1 模型的优点

1.

2.

3.

6.2 模型的不足

1.

2.

3.

6.3 模型的推广与应用

1.

2.

3.

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于 [简要用途, 如语言润色, 代码调试等]，详细使用情况见支撑材料。

AI 工具使用详情

工具信息

项目	填写内容
工具名称	填写实际使用的产品或工具名称
版本或型号	填写实际版本、模型名称或可识别的型号
使用日期	*YYYY-MM-DD* 至 *YYYY-MM-DD*

具体使用目的和环节

逐项说明 AI 用于语言润色、代码调试、资料整理或其他环节；不得笼统写“辅助论文”。明确其输入、输出及对应论文位置。

主要提示方式与使用过程

说明主要提示策略与迭代过程，可附具有代表性的交互片段。不要包含参赛者姓名、学校或赛区信息。

采纳、人工修改与核验

除语言润色外，逐项记录 AI 输出是否采纳、由谁进行何种人工修改、采用何种数据或实验复核，以及最终结论是否发生变化。

人工核验清单

1) 核对公式推导、单位、边界条件和符号一致性；
2) 在独立环境中运行全部代码并复现论文表图；
3) 核验引用、数据来源和事实陈述；
4) 确认核心建模与分析由参赛队主导；
5) 确认论文声明与本详情文件一致。

参考文献

[1] 刘海洋. LaTeX 入门[J]. 电子工业出版社, 北京, 2013.

[2] 全国大学生数学建模竞赛论文格式规范 (2023 年修改).

[1] https://www.latexstudio.net

附录

A 模板所用的宏包

表 4 宏包罗列

模板中已经加载的宏包				
amsbsy	amsfonts	amsgen	amsmath	amsopn
amssymb	amstext	appendix	array	atbegshi
atveryend	auxhook	bigdelim	bigintcalc	bigstrut
bitset	bm	booktabs	calc	caption
caption3	CJKfntef	cprotect	ctex	ctexhook
ctexpatch	enumitem	etexcmds	etoolbox	everysel
expl3	fix-cm	fontenc	fontspec	fontspec-xetex
geometry	gettitlestring	graphics	graphicx	hobsub
hobsub-generic	hobsub-hyperref	hopatch	hxetex	hycolor
hyperref	ifluatex	ifpdf	ifthen	ifvtex
ifxetex	indentfirst	infwarerr	intcalc	keyval
kvdefinekeys	kvoptions	kvsetkeys	l3keys2e	letltxmacro
listings	longtable	lstmisc	ltcaption	ltxcmds
multirow	nameref	pdfescape	pdftexcmds	refcount
rerunfilecheck	stringenc	suffix	titletoc	tocloft
trig	ulem	uniquecounter	url	xcolor
xcolor-patch	xeCJK	xeCJKfntef	xeCJK-listings	xparse
xtemplate	zhnumber			

B 代码一

```
kk=2;[mdd,ndd]=size(dd);
while ~isempty(V)
[tmpd,j]=min(W(i,V));tmpj=V(j);
for k=2:ndd
[tmp1,jj]=min(dd(1,k)+W(dd(2,k),V));
tmp2=V(jj);tt(k-1,:)=[tmp1,tmp2,jj];
end
tmp=[tmpd,tmpj,j;tt];[tmp3,tmp4]=min(tmp(:,1));
if tmp3==tmpd, ss(1:2,kk)=[i;tmp(tmp4,2)];
else,tmp5=find(ss(:,tmp4)~=0);tmp6=length(tmp5);
```

```
if dd(2,tmp4)==ss(tmp6,tmp4)
ss(1:tmp6+1,kk)=[ss(tmp5,tmp4);tmp(tmp4,2)];
else, ss(1:3,kk)=[i;dd(2,tmp4);tmp(tmp4,2)];
end;end
dd=[dd,[tmp3;tmp(tmp4,2)]];V(tmp(tmp4,3))=[];
[mdd,ndd]=size(dd);kk=kk+1;
end; S=ss; D=dd(1,:);

```

C 代码二

```
kk=2;
[mdd,ndd]=size(dd);
while ~isempty(V)
    [tmpd,j]=min(W(i,V));tmpj=V(j);
for k=2:ndd
    [tmp1,jj]=min(dd(1,k)+W(dd(2,k),V));
    tmp2=V(jj);tt(k-1,:)=[tmp1,tmp2,jj];
end
    tmp=[tmpd,tmpj,j;tt];[tmp3,tmp4]=min(tmp(:,1));
if tmp3==tmpd, ss(1:2,kk)=[i;tmp(tmp4,2)];
else,tmp5=find(ss(:,tmp4)~=0);tmp6=length(tmp5);
if dd(2,tmp4)==ss(tmp6,tmp4)
    ss(1:tmp6+1,kk)=[ss(tmp5,tmp4);tmp(tmp4,2)];
else, ss(1:3,kk)=[i;dd(2,tmp4);tmp(tmp4,2)];
end;
end
    dd=[dd,[tmp3;tmp(tmp4,2)]];V(tmp(tmp4,3))=[];
    [mdd,ndd]=size(dd);
    kk=kk+1;
end;
S=ss;
D=dd(1,:);

```